

Symbolica

Computer algebra for Typst

User manual · version 0.2.0

Symbolic algebra, calculus, and numerical evaluation in Typst.

Free for all use within Typst, including commercial work. No Symbolica license key is required.

symbolica.io · Repository · Typst permission

Contents

1 Introduction 3
1.1 Installation 3
2 Expressions and symbols 3
2.1 Create an expression 3
2.2 Parse formulas and expression strings 4
2.3 Substitute a value or evaluate a formula 4
2.4 Variables, functions, and wildcards 4
2.5 Differentiate built-in functions 6
2.6 Use special functions 6
2.7 Write equations as expressions equal to zero 7
3 Worked examples 7
3.1 Derive and fit a pendulum model 7
3.2 Rewrite a trigonometric identity 8
3.3 Cancel factors and take partial fractions 9
3.4 Find the intersections of a circle and a line 9
3.5 Interpolate with a matrix 10
3.6 Sample a surface and its gradient 10
4 Symbolic integration 11
4.1 Compute an antiderivative 11
4.2 Inspect a worked rule trace 11
4.3 Step fields 12
5 Limitations 12
5.1 Troubleshooting 13
6 API reference 13
6.1 Parsing, symbols, and rendering 13
6.2 Built-in special functions 21
6.3 Algebra and calculus 25
6.4 Rewriting 32
6.5 Evaluation and solving 38
6.6 Matrices 43
6.7 Scalar constructors 52
6.8 Document helpers 54
6.9 Advanced configuration 54
7 Namespaces and parser grammars 55
7.1 Keep symbol names separate 55
7.2 Define a parser grammar 56
8 Compatibility and licensing 58
9 Acknowledgements 58

1 Introduction

Symbolica is a computer algebra system used in research, available as a Rust crate and through Python bindings. This plugin runs it as WebAssembly inside Typst, so you can factor polynomials, take derivatives, solve equations, and evaluate the results directly in your document.

When writing a derivation, you often expand an expression, substitute a definition, or isolate a variable. You can write these operations in Typst and let Symbolica calculate each step. Change the starting expression, and the displayed results update with it.

`parse` reads a formula and `to-typst` displays a result. For example, expand and factor a polynomial:

```
#import "@preview/symbolica:0.2.0": *

#let p = parse($(x + y)^3 - (x^3 + y^3)$)
$ #to-typst(factor(expand(p))) $
```

$$3xy(x+y)$$

Free within Typst
The plugin is free for all use within Typst, including commercial work. No license key is needed.

Symbolica itself is source-available. Outside Typst, its standard terms allow one core and one instance per device for free non-commercial use; commercial work requires a professional license. The plugin's Typst permission covers its use here.

1.1 Installation

Import Symbolica from Typst Universe. Typst downloads and caches the package automatically:

```
#import "@preview/symbolica:0.2.0": *
```

Use the wildcard import for ordinary calculations. The API names leave Typst's native `math`, `symbol`, `function`, `content`, and math layout functions available. Import as a module if you prefer qualified names.

2 Expressions and symbols

2.1 Create an expression

`parse` turns a formula into a symbolic expression. Store it in a variable, apply an operation, and use `to-typst` to show the result:

```
#let f = parse($(x + 1)^3$)
#let expanded = expand(f)

$
  (x + 1)^3 &= #to-typst(expanded)
$
```

$$(x+1)^3 = 1 + 3x + 3x^2 + x^3$$

The expression `f` keeps its original form; `expand(f)` returns a new value. Use `factor` to look for a product and `collect` to group terms by powers of a chosen variable.

Some arithmetic happens automatically. Symbolica combines like terms and reduces fractions when you create an expression:

```
#let f = parse($2 x + x/2 + 1/3 + 1/6$)
$ #to-typst(f) $
```

$$\frac{1}{2} + \frac{5}{2}x$$

Integers and fractions remain exact. Decimal input such as `0.1` is stored as a floating-point number. Expansion and factorization require explicit calls; automatic normalization does not choose a single form for every algebraically equivalent expression.

For plain text, use `canonical`; for LaTeX source, use `to-latex`. Symbolica's expression guide explains its normal form in more detail.

2.2 Parse formulas and expression strings

Use Typst math when writing a formula in your document. Use a string when reading an expression from data or another program:

```
#let from-math = parse($x^2 / (1 + x)$)
#let from-string = parse("x^2/(1+x)")
$ #to-typst(from-string) $
```

$$\frac{x^2}{1+x}$$

Strings use Symbolica's syntax, including explicit multiplication: `parse("2*x + y")`. Typst math allows juxtaposition: `parse($2 x + y$)`. The string parser does not evaluate Typst code.

`parse` also accepts integers, floats, and existing expressions. Integers stay exact; floats keep their value. Existing Atom bytes pass through unchanged, including their symbol identities, regardless of `namespace`.

Algebra functions accept numbers and single-leaf strings directly. For a whole expression in a string, call `parse` first: `expand(parse("(x+1)^2"))`.

To create one variable, use `literal("x")`. To give symbols separate names or change the interpretation of Typst math, see Section 7.

2.3 Substitute a value or evaluate a formula

`replace` substitutes an expression for a variable and keeps the answer symbolic. `evaluate` calculates a numerical value once you supply values for the remaining variables:

```
#let x = literal("x")
#let f = parse($x^2 + 2 x$)
#let shifted = replace(f, x, parse($t + 1$))
#let value = evaluate(f, values: ((x, 2.0),))

$ f(t + 1) = #to-typst(shifted) $\
$ f(2) = #value.re $
```

$$f(t+1) = 2(1+t) + (1+t)^2$$

$$f(2) = 8$$

`values` is an array of `(variable, value)` pairs. The result has fields `re` and `im` for its real and imaginary parts. Here the imaginary part is zero. For several input points, use `evaluate-many`; for a Cartesian grid, use `evaluate-grid`. The sampling example in Section 3.6 shows both the variable order and the shape of the returned data.

2.4 Variables, functions, and wildcards

`literal("x")` creates a variable that can be differentiated or substituted. `function-head("g")` creates a callable name for an unspecified function:

```
#let x = literal("x")
#let g = function-head("g")
#let f = parse($#g(x) + x$)
$ #to-typst(replace(f, x, parse($y + 1$))) $
```

$$1 + y + g(1+y)$$

Substitution reaches the arguments of `g` as well as the separate x. Declaring a function gives it a symbolic identity; numerical evaluation also needs to know what the function computes.

A wildcard stands for an expression to be matched. In this rule, `a` takes the argument of each call to `g`:

```
#let g = function-head("g")
#let a = wild("a")
#let squared = rule(g(a), pow(a, 2))
#let f = parse($#g(2) + #g(5)$)

$ #to-typst(replace-multiple(f, (squared,))) $
```

$$29$$

`wild("a")` creates the pattern symbol `a_`; give the base name without the underscore. Repeated occurrences of the same wildcard in one pattern must match the same expression. The trigonometric example in Section 3.2 uses this to require equal arguments for sine and cosine.

2.4.1 Literal display labels

Sometimes a whole label should act as one variable. Pass math or text to `literal` to keep that label intact. For example, `literal($x+1$)` is one symbol, whereas `parse($x+1$)` adds two expressions. Replacing x has no effect inside a literal label:

```
#let label = literal($x + 1$)
$ #to-typst(pow(label, 2)) $
```

$$(x + 1)^2$$

Both renderers group composite labels so their boundaries stay clear in products and powers. `to-typst` retains exact Atom metadata. `to-typst-source` preserves appearance, but its plain text does not preserve the opaque identity.

A namespace groups symbol names: two groups can each have their own x. See Section 7.1 for examples. Without an explicit name, the same display in the same namespace creates the same symbol. Simple `literal($x$)` and `literal("x")` agree, as do `literal($a_0$)` and `parse($a_0$)`. A `name` can be supplied only for content; it selects a semantic name independently of the label:

```
#let first = literal(
  $a_0$, name: "first_coefficient", namespace: "model",
)
#let second = literal(
  $a_0$, name: "second_coefficient", namespace: "model",
)
$ #to-typst(subtract(first, second)) $
```

$$a_0 - a_0$$

The difference does not cancel: the labels look alike, but the names are different. Declare a named label before using that name in other expressions. Literal labels support math and text, including attachments. For custom styling or layout, keep a simple symbol name and use `notation`.

2.4.2 Preserve symbol identity in a formula

Use `#` to insert a declared symbol into Typst math. Its namespace, tags, and display label travel with it:

```
#let mass = literal("m", namespace: "model", tags: ("model::mass",))
#let shell = parse($p^2 - #mass^2$)

$ p^2 - m^2 = #to-typst(shell) $
```

$$p^2 - m^2 = -m^2 + p^2$$

`parse` reads the inserted symbol's identity, including its `model` namespace. The tag `model::mass` is a label for use by notation rules or other packages; it does not by itself impose a mathematical assumption. Declare symbols with tags before parsing expressions that use them.

`function-head` also accepts a namespace and tags. Inserting a call preserves both its head and its arguments:

```
#let response = function-head(
  "R", namespace: "model", tags: ("model::response",),
)
#let time = literal("t", namespace: "model")
#let value = parse($#response(time) + 1$)

$ R(t) + 1 = #to-typst(value) $
```

$$R(t) + 1 = 1 + R(t)$$

2.4.3 Choose how an expression is displayed

Use `notation` to change a symbol's label or the layout of a function call. Here `x` displays as ξ, and `f(x+1)` displays with a calligraphic letter and square brackets:

```
#let x = literal("x", namespace: "notation_example")
#let f = function-head("f", namespace: "notation_example")
#let expression = f(add(x, 1))
#let display = notation(
  heads: ("notation_example::x": $xi$),
  calls: (
    "notation_example::f": ctx => {
      let (argument,) = ctx.visual-arguments
      $cal(F)[#argument]$
    },
  ),
)

$ #to-typst(expression, notation: display) $
```

$$\mathcal{F}[1 + \xi]$$

`heads` maps full symbol names to labels. A `calls` renderer receives the call's arguments through `ctx.visual-arguments`. The displayed result retains its mathematical identity, so `parse` can read it back. `to-typst-source` returns source text with the same default layout, but plain text cannot carry this hidden identity.

2.5 Differentiate built-in functions

Symbolica knows derivatives of functions such as sine, cosine, and the exponential. Select its built-in namespace when parsing them from Typst math:

```
#let x = literal("x")
#let f = parse($exp(#x) sin(#x)$, namespace: "symbolica")
$ f'(x) = #to-typst(derivative(f, x)) $
```

$$f'(x) = e^{x} \sin(x) + e^{x} \cos(x)$$

The inserted `#x` retains the identity of the declared variable. The namespace selects Symbolica's function definitions; Section 7.1 explains how to set it for a whole calculation.

2.6 Use special functions

For functions that Typst does not name directly, use a constructor such as `polylog` or `bessel-j`. These always select Symbolica's built-in function:

```
#let x = literal("x")
#let f = polylog(2, x)
#let value = evaluate(polylog(2, divide(1, 2)))

$ f(x) = #to-typst(f) $\
$ f'(x) = #to-typst(derivative(f, x)) $\
$ f(1/2) approx #calc.round(value.re, digits: 8) $
```

$$f(x) = \operatorname{polylog}(2, x)$$

$$f'(x) = \frac{-\log(1 - x)}{x}$$

$$f\left(\frac{1}{2}\right) \approx 0.58224053$$

The constructor's result can go directly into algebra and evaluation calls, or into a larger formula: `parse($#polylog(2, x) + 1$)`.

The direct constructors are:

Function	Constructor	Meaning
Gamma	`gamma-function(z)`	$\Gamma(z)$
Polygamma	`polygamma(n, z)`	Order n, with nonnegative integer n; $n = 0$ is digamma.
Polylogarithm	`polylog(s, z)`	Order s, argument z.
Riemann zeta	`zeta-function(s)`	$\zeta(s)$

Function	Constructor	Meaning
Bessel	`bessel-j(nu, z)`, `bessel-y(nu, z)`	$J_{\nu(z)}$ and $Y_{\nu(z)}$.
Modified Bessel	`bessel-i(nu, z)`, `bessel-k(nu, z)`	$I_{\nu(z)}$ and $K_{\nu(z)}$.

Use `evaluate` for a numerical value, then choose how many digits to display:

```
$ J_0(1) approx #calc.round(evaluate(bessel-j(0, 1)).re, digits: 8) $\
$ zeta(3) approx #calc.round(evaluate(zeta-function(3)).re, digits: 8) $
```

$$J_0(1) \approx 0.76519769$$

$$\zeta(3) \approx 1.2020569$$

For other built-ins, use `function-head(name, namespace: "symbolica")` with Symbolica's spelling, such as `"bessel_j"`. An unknown name creates an unspecified symbolic function. It will not acquire a numerical definition from its name alone. Supply numerical values for every free variable before calling `evaluate`. Some derivatives and series of special functions are not implemented; exact fractional arguments also avoid the decimal-input issue described under Section 5.

2.7 Write equations as expressions equal to zero

Solver inputs are expressions understood to equal zero. For example, `parse($x + y - 3$)` represents $x + y = 3$.

Pass the unknowns in the order you want them returned. `solve` returns one entry in `branches` for each solution; its `values` array follows that order. The worked example in Section 3.4 compares these exact branches with numerical solutions from two starting points.

With symbolic parameters, inspect `coverage`: a `"generic"` result applies where the expressions in `coverage-guard` are nonzero. `nsolve` and `nsolve-system` instead search from a supplied starting point and may fail to converge or find a different root when that point changes.

3 Worked examples

3.1 Derive and fit a pendulum model

A pendulum has potential $V(\theta) = \kappa(1 - \cos\theta)$. Suppose a torque sensor also has an unknown offset τ_0. We will derive a small-angle model, then fit κ and τ_0 to two measurements. This follows Symbolica's pendulum-calibration example.

3.1.1 Differentiate and expand

The restoring torque is $-\partial\frac{V}{\partial}\theta$. Keep terms through θ^3 to include the first correction to the linear approximation.

```
#let angle = literal($theta$)
#let stiffness = literal($kappa$)
#let offset = literal($tau_0$)
#let potential = parse(
  $#stiffness (1 - cos(#angle))$
)
#let torque = neg(derivative(potential, angle))
#let model = add(series(torque, angle, 0, 3), offset)

$ tau(theta) = #to-typst(torque) $\
$ tau_"model"(theta) = #to-typst(model) $
```

$$\tau(\theta) = -\kappa\sin(\theta)$$

$$\tau_{\mathrm{model}(\theta)} = -\theta\kappa + \tau_0 + \frac{1}{6}\theta^3\kappa$$

`series` returns the truncated expression; it does not include an order term. This model is approximate in θ but linear in its unknown parameters.

3.1.2 Solve for the parameters

Let the measurements be (θ_1, τ_1) and (θ_2, τ_2). Substitute each angle, subtract the measured torque, and pass both residuals to `solve`. Each input expression is understood to equal zero.

```
#let angle1 = literal($theta_1$)
#let angle2 = literal($theta_2$)
#let torque1 = literal($tau_1$)
#let torque2 = literal($tau_2$)
#let equations = (
  subtract(replace(model, angle, angle1), torque1),
  subtract(replace(model, angle, angle2), torque2),
)
#let solution = solve(equations, (stiffness, offset))
#let fit = solution.branches.first().values

$ kappa = #to-typst(fit.at(0)) $\
$ tau_0 = #to-typst(fit.at(1)) $
```

$$\kappa = \frac{6\tau_1 - 6\tau_2}{-6\theta_1 + 6\theta_2 + \theta_1^3 - \theta_2^3}$$

$$\tau_0 = \frac{-6\theta_1\tau_2 + 6\theta_2\tau_1 + \theta_1^3\tau_2 - \theta_2^3\tau_1}{-6\theta_1 + 6\theta_2 + \theta_1^3 - \theta_2^3}$$

`values` follows the variable order passed to `solve`: stiffness first, then offset. These formulas require distinct values of the angle-dependent coefficient; two identical readings cannot determine both parameters.

3.1.3 Evaluate at the measurements

For readings $(0.10, -0.4697)$ and $(0.20, -0.9545)$, evaluate both fitted parameters in one call. `evaluate-many` returns a row per input point and a complex value for each expression; `.re` reads the real part.

```
#let fitted = evaluate-many(
  fit, (angle1, angle2, torque1, torque2),
  ((0.10, 0.20, -0.4697, -0.9545),),
).first()
#let (k, b) = fitted.map(value => value.re)
#let predictions = evaluate-many(
  model, (angle, stiffness, offset),
  ((0.10, k, b), (0.20, k, b)),
)

$ kappa approx #calc.round(k, digits: 6) $\
$ tau_0 approx #calc.round(b, digits: 6) $\
Predicted torques:
#predictions.map(row =>
  str(calc.round(row.first().re, digits: 4))
).join(", ")
```

$$\kappa \approx 4.905228$$

$$\tau_0 \approx 0.020005$$

```
Predicted torques: -0.4697,
-0.9545
```

The predictions reproduce the two readings. More measurements would let us test whether the cubic approximation describes the data well.

3.2 Rewrite a trigonometric identity

To use $\sin^2(a) + \cos^2(a) = 1$ with any argument, represent a by a wildcard. Both occurrences must capture the same expression. This is the repeated wildcard rule described in Symbolica's pattern-matching guide.

```
#let sine = function-head("sin", namespace: "symbolica")
#let cosine = function-head("cos", namespace: "symbolica")
#let a = wild("a")
#let pattern = parse($#sine(a)^2 + #cosine(a)^2$)
#let source = parse(
  $3 (#sine("x")^2 + #cosine("x")^2)
   + #sine($x+y$)^2 + #cosine($x+y$)^2$
)
#let result = replace(source, pattern, 1, repeat: true)

$ #to-typst(source) = #to-typst(result) $
```

$$3\left(\sin(x)^2 + \cos(x)^2\right) + \sin(x+y)^2 + \cos(x+y)^2 = 4$$

The two matches capture x and $x+y$. `repeat: true` runs further passes until nothing changes. Use it only for rules that terminate: a rule that changes a into $a+1$ would run forever.

Matching follows the expression's structure. For example, x^2 occurs in `expand(parse($(x+1)^2$))`, but is not a subexpression of the unexpanded square.

3.3 Cancel factors and take partial fractions

This rational function contains a removable factor $s+3$. After cancellation, `apart` separates it into terms with simple poles. `together` combines those terms over a common denominator.

```
#let s = literal("s")
#let response = parse(
  $((s+3)(2s+5)) / (s^3+6s^2+11s+6)$
)
#let reduced = cancel-factors(response)
#let fractions = apart(reduced, s)
#let residual = together(subtract(fractions, reduced))

$ H(s) &= #to-typst(reduced) \
       &= #to-typst(fractions) $\
Difference: $#to-typst(residual)$
```

$$\begin{aligned} H(s) &= \frac{5+2s}{2+3s+s^2} \\ &= \frac{3}{1+s} + \frac{-1}{2+s} \end{aligned}$$

Difference: 0

The remaining poles are $s=-1$ and $s=-2$. Cancellation does not restore the original function's value at $s=-3$: that input was excluded by its denominator.

3.4 Find the intersections of a circle and a line

The equations $x^2+y^2=25$ and $x-y=1$ have two intersections. Write each with zero on the right and ask `solve` for all real branches.

```
#let x = literal("x")
#let y = literal("y")
#let system = (
  parse($x^2 + y^2 - 25$),
  parse($x - y - 1$),
)
#let solutions = solve(system, (x, y), domain: "real")

#for branch in solutions.branches [
  $ x = #to-typst(branch.values.at(0)),
    quad y = #to-typst(branch.values.at(1)) $\
]
```

$$x = 4, \quad y = 3$$

$$x = -3, \quad y = -4$$

A numerical solve instead starts from one point and searches for a root. These two starting points converge to different intersections.

```
#let roots = ((3.0, 3.0), (-3.0, -3.0)).map(start =>
  nsolve-system(system, (x, y), start, prec: 1e-10)
)
#let residuals = evaluate-many(system, (x, y), roots)

#for root in roots [
  $#calc.round(root.at(0), digits: 6),
    quad #calc.round(root.at(1), digits: 6)$\
]
```

$$4, \quad 3$$
$$-3, \quad -4$$

`residuals` evaluates both equations at each root; its entries should be close to zero. Unlike the exact solve above, a numerical solve does not enumerate all solutions, and another starting point may fail to converge.

3.5 Interpolate with a matrix

Find the quadratic $p(t) = a_0 + a_1 t + a_2 t^2$ through $(0, 1)$, $(1, 3)$, and $(2, 8)$. Each point supplies a row of $Aa = b$. `matrix-solve` keeps the coefficients exact.

```
#let A = matrix(((1, 0, 0), (1, 1, 1), (1, 2, 4)))
#let b = vector((1, 3, 8))
#let coefficients = matrix-solve(A, b)
#let (a0, a1, a2) = range(3).map(i =>
  to-typst(matrix-at(coefficients, i, 0))
)
#let polynomial = parse($#a0 + #a1 t + #a2 t^2$)

$ a = #to-typst(coefficients) $\
$ p(t) = #to-typst(polynomial) $
```

$$a = \begin{pmatrix} 1 \\ \frac{1}{2} \\ \frac{3}{2} \end{pmatrix}$$

$$p(t) = 1 + \frac{1}{2}t + \frac{3}{2}t^2$$

`matrix-at` uses zero-based row and column indices. Multiplying Aa returns b, and the nonzero determinant confirms that this interpolant is unique. For larger symbolic systems, Symbolica's matrix documentation describes the underlying exact arithmetic.

3.6 Sample a surface and its gradient

For $f(x, y) = x^2 + xy + y^2$, differentiate first, then evaluate the height and both gradient components over a grid. This avoids approximating derivatives from sampled values.

```
#let x = literal("x")
#let y = literal("y")
#let f = parse($x^2 + x y + y^2$)
#let fx = derivative(f, x)
#let fy = derivative(f, y)
#let grid = evaluate-grid(
  (f, fx, fy), (x, y),
  (domain(-1, 1, samples: 3), domain(-1, 1, samples: 3)),
)

$ partial_x f = #to-typst(fx) $\
$ partial_y f = #to-typst(fy) $
```

$$\partial_x f = 2x + y$$

$$\partial_y f = x + 2y$$

x	y	f	$\partial_x f$	$\partial_y f$
−1	−1	3	−3	−3
−1	0	1	−2	−1
−1	1	1	−1	1
0	−1	1	−1	−2
0	0	0	0	0
0	1	1	1	2
1	−1	1	1	−1
1	0	1	2	1
1	1	3	3	3

Each row of `grid.values` corresponds to the same row of `grid.points`, with values ordered as `(f, fx, fy)`. The last axis varies fastest. For an irregular set of points, pass those points to `evaluate-many` instead.

4 Symbolic integration

Symbolica uses Rubi's integration rules to find antiderivatives. You can pass the same expressions and variables used for algebra and differentiation.

4.1 Compute an antiderivative

Find an antiderivative with `integrate(expression, variable)`. Here we integrate $\frac{x}{x+1}$ and check the answer by differentiating it:

```
#let x = literal("x")
#let f = parse($x / (x + 1)$)
#let primitive = integrate(f, x)
#let check = together(subtract(derivative(primitive, x), f))

$ integral #to-typst(f) dif x = #to-typst(primitive) + C $\
$ F'(x) - f(x) = #to-typst(check) $
```

$$\int \frac{x}{1+x}\,\mathrm{d}x = x - \log(1+x) + C$$

$$F'(x) - f(x) = 0$$

The integration variable must be a single symbol. Add the integration constant C when displaying an indefinite integral; it is not included in the returned expression. Preserve any domain restrictions in your derivation.

> **First integration call**
> The first call to `integrate` or `integrate-with-steps` compiles and initializes the rule sets, which may take about 10 seconds. Both functions share the cached rules, so later calls and ordinary edits reuse them. Restarting the compiler or clearing its cache repeats this setup.

4.2 Inspect a worked rule trace

Use `integrate-with-steps` when you want to show how Rubi transformed the integral. `result` holds the antiderivative and `steps` holds the sequence of rule applications. The example below displays each transformation and indents the integrals introduced by earlier steps:

```
#let x = parse($x$)
#let f = parse($x / (x + 1)$)
#let explanation = integrate-with-steps(f, x)
#let step-notation = notation(
  calls: (
    "symbolica_integrate::rubi_int": ctx => {
      let (body, variable) = ctx.visual-arguments
      $ integral #body dif #variable $
    },
  ),
)
#let show-step(expression) = to-typst(
  expression, notation: step-notation,
)

$ integral #to-typst(f) dif x = #to-typst(explanation.result) + C $

#for step in explanation.steps [
  #h(step.depth * 1.15em)
  #if step.rule == none [*Transformation*] else [*Rule #step.rule*]
  #if step.description != "" [: #step.description]
  #linebreak()
  #h(step.depth * 1.15em)
  $#show-step(step.input) = #show-step(step.output)$
  #linebreak()
]
```

$$\int \frac{x}{1+x}\,\mathrm{d}x = x - \log(1+x) + C$$

Rule 49 : Algebraic expansion
$\int \frac{x}{1+x}\,\mathrm{d}x = \int 1 + \frac{-1}{1+x}\,\mathrm{d}x$
  Rule 2009 : Integrate the terms of the sum separately.
  $\int 1 + \frac{-1}{1+x}\,\mathrm{d}x = \int 1\,\mathrm{d}x + \int \frac{1}{-1-x}\,\mathrm{d}x$
    Rule 24 : Apply the direct antiderivative formula.
    $\int 1\,\mathrm{d}x = x$
    Rule 16 : Apply the direct antiderivative formula.
    $\int \frac{-1}{1+x}\,\mathrm{d}x = -\log(1+x)$

Check `complete` before treating the result as a finished antiderivative. If it is `false`, some integrals remain unevaluated. Each step's `input` and `output` is a symbolic expression that you can display with `to-typst` or use in another calculation.

4.3 Step fields

Each step contains:

- `rule`: the Rubi rule number, or `none` for an auxiliary transformation;
- `depth`: nesting depth of the rewritten integral;
- `description`, `references`, and `source`: Rubi's explanatory metadata; and
- `input` and `output`: portable Atom payload bytes for the immediate rewrite.

Steps run from an outer transformation into its nested integrals. Use `depth` to show that nesting and `rule` to identify the Rubi rule applied.

5 Limitations

The plugin covers the operations in this manual. When a calculation needs more, Symbolica's Python and Rust interfaces expose additional algorithms and configuration options. In Typst, keep these limits in mind:

- Integration returns a best-effort antiderivative without an integration constant. Unsupported parts remain unevaluated; inspect `complete` in `integrate-with-steps` to check whether the entire expression was integrated.
- Exact system solving is intended for linear and polynomial equations. Numerical solving needs a starting point. Check the answer by substituting it into the original equations.
- Decimal literals remain floating-point values. A current upstream Wasm bug can mis-evaluate them inside analytic functions. Write exact fractions for those inputs—for example, `cos(1/2)`—before numerical evaluation.
- `to-typst` can omit digits or misplace the decimal point in floating-point results. Use `evaluate` and display its `re` and `im` fields, or use `canonical` for a text representation of an approximate expression.
- Matrix entries must be rational-polynomial expressions, and their dimensions must agree for the requested operation.
- Algebraic transformations do not keep a separate list of assumptions. If a manipulation is valid only under a condition such as $x \neq 1$, preserve that condition in the surrounding document.
- Some unusual Typst math structures may not parse. `array-tree` can help show what the parser received.

5.1 Troubleshooting

Symptom	Likely cause	What to try
`expected content, found bytes`	A symbolic result was inserted directly into `$...$`.	Display it with `to-typst`.
A symbolic result fails in another API	The bytes hold a matrix or came from an incompatible package version.	Use the matching package version and keep matrix values in matrix APIs.
A function cannot be evaluated numerically	It has no numerical definition, or an argument still contains a free variable.	Use a built-in constructor and supply all variable values to `evaluate`.
An analytic function of a decimal gives an unexpected value	Its floating-point argument encountered the current upstream Wasm bug.	Use an exact fraction such as `cos(1/2)`, then call `evaluate`.
A replacement does not match	Repeated wildcards would have to capture different expressions.	Check that every occurrence of the wildcard should match the same value.
An exact solver errors	An equation was not rearranged to zero, or the system is outside the supported polynomial scope.	Move every term to the left; try `nsolve-system` for a numerical branch.
A numerical solver errors or finds an unwanted root	The starting point leads to a different root or no root.	Try another physically meaningful initial guess and check residuals.
A matrix operation errors	Shapes differ, a matrix is singular, or an entry is unsupported.	Inspect `matrix-shape`, `determinant`, and the coefficient expressions.

6 API reference

This section lists all public functions, grouped by operation. It is generated from the documentation in `lib.typ`.

6.1 Parsing, symbols, and rendering

6.1.1 parse

Convert math content, an expression string, or a number into an Atom payload.

Arithmetic, fractions, powers, roots, absolute values, calls, and common Typst math structures are translated through the configured grammar. Matrix-valued `mat(...)` and `vec(...)` content must instead be passed to `matrix` or `vector`. Keep the returned bytes opaque and use this module's functions to inspect or transform them. Decimal literals remain floating-point coefficients; write a fraction when you need exact input. Integers become exact numbers, floats keep their floating-point value, and existing Atom payload bytes pass through unchanged. A

namespace override does not rename symbols in an existing payload. Matrix payloads belong in the matrix APIs and are not scalar expressions.

Strings use Symbolica expression syntax, for example `"x^2/(1+x)"` or `"f(x, y)"`. They are passed directly to Symbolica without evaluating Typst source. Symbolica recognizes built-in names such as `sin`, `gamma`, and `pi`; other unqualified names use the requested namespace. A `grammar` override applies only to Typst content and is rejected for other inputs. The engine's configured grammar also affects only content. String arguments passed directly to algebra operations accept single leaves: numbers or valid symbol names. They do not parse arithmetic expressions, and invalid symbol names are rejected. Use `parse("x+1")` to read an expression from a string.

Subscripts, primes, and corner attachments identify decorated symbols. Their contents retain display order: `C_(i j)` differs from `C_(j i)`. Superscripts remain algebraic powers, including `a_0^2`. Decoration is opaque to algebra: replacing `i` does not modify the label in `C_(i j)`, and `h'(c)` names a decorated function without implying a derivative. Use `h_(i)(c)` to keep a subscript separate from the function arguments.

```
#let expr = parse($x + 1$)
#to-typst(expr)
#to-typst(parse("x^2/(1+x)"))
```

$1 + x$ $\frac{x^2}{1+x}$

Parameters

```
parse(
  input: bytes content int float str,
  grammar: dictionary none,
  namespace: str none
) -> bytes
```

input `bytes` or `content` or `int` or `float` or `str`

Math content (`$...$`), a Symbolica expression string, a number, or an existing Atom payload.

grammar `dictionary` or `none`

Typst content grammar override. `none` uses the engine grammar. An explicit grammar is rejected for strings, numbers, and existing payloads.

Default: `none`

namespace `str` or `none`

Namespace for parsed symbols. `none` uses the engine namespace (`"typst"` for the top-level function).

Default: `none`

6.1.2 literal

Construct an ordinary Symbolica symbol with an optional portable label.

A string supplies the symbol name. Math or text content supplies an opaque label stored in Symbolica symbol data: `literal($x+y$)` is one variable, while `parse($x+y$)` is a sum. Replacing `x` does not change that label. Labels retain ordered attachments and are grouped when needed for display.

Equivalent labels in one namespace share an identity. `literal($x$)` equals `literal("x")`, and `literal($a_0$)` equals `parse($a_0$)`. With `name`, the supplied name determines identity; different names may share one label. Declare a named label before using that name elsewhere. Conflicting labels or tags for the same name are rejected.

The returned content carries exact Atom metadata, so interpolation and reparsing `to-typst` retain the whole symbol. Plain `to-typst-source` output preserves its appearance but carries no semantic metadata.

Portable labels support math elements and plain text. Unsupported styling, layout, contextual content, or element fields produce an error. Use a named string literal and a document-side `notation` renderer for custom content. Names ending with _ are reserved for `wild` and are rejected here.

```
#let label = literal($x+y$)
#to-typst(pow(label, 2))
#let x = literal("x", namespace: "model", tags: ("model::positive",))
#to-typst(parse($#x^2 + 1$))
```

$(x + y)^2 \quad 1 + x^2$

Parameters

```
literal(
  input: str content,
  name: str none,
  namespace: str none,
  tags: array
) -> content
```

input `str` or `content`

Symbol-name string or opaque mathematical/text label.

name `str` or `none`

Explicit identity for a content label; `none` derives identity from it.

Default: `none`

namespace `str` or `none`

Namespace override. `none` uses the engine namespace.

Default: `none`

tags `array`

Portable semantic labels retained in Symbolica symbol data.

Default: `()`

6.1.3 function-head

Construct a callable Symbolica function with semantic Typst metadata.

Calling the returned Typst function constructs the exact Symbolica call, prints that Atom, and attaches one metadata envelope to the complete call. Interpolate callable bindings inside math, for example `#f(x)`.

Use `namespace: "symbolica"` for built-in special functions, even when Typst has no built-in function of that name. For example, `function-head("polylog", namespace: "symbolica")(2, x)` constructs a polylogarithm. Direct constructors are also available: `gamma-function`, `polygamma`, `polylog`, `zeta-function`, `bessel-j`, `bessel-y`, `bessel-i`, and `bessel-k`. Unknown names create ordinary symbolic functions; they do not acquire a numerical implementation.

```
#let f = function-head(
  "f", namespace: "model", tags: ("model::smooth",),
)
#to-typst(parse($#f(literal("x")) + 1$))
```

$1 + f(x)$

Parameters

```
function-head(
  name: str,
  namespace: str none,
  tags: array
) -> function
```

name `str`

Function-head name.

namespace `str` or `none`

Namespace override. `none` uses the engine namespace.

Default: `none`

tags `array`

Portable semantic labels retained in the attached metadata.

Default: `()`

6.1.4 wild

Construct a Symbolica pattern wildcard.

A wildcard captures part of a matching expression. Repeating the same wildcard in a pattern requires equal captures. Its Symbolica name is `name` followed by `level` underscores; use `literal` for an algebraic variable.

```
#let a = to-typst(wild("a"))
#let pattern = parse($f(#a, #a)$)
#to-typst(replace(parse($f(x, x) + f(x, y)$), pattern, 1))
```

$1 + f(x, y)$

Parameters

```
wild(
  name: str,
  level: int,
  namespace: str none
) -> bytes
```

name `str`

Base name of the wildcard, without trailing underscores.

level `int`

Positive number of underscore levels to append. `1` creates a conventional single wildcard such as `a_`. Zero is rejected; use `literal` for a symbol.

Default: 1

namespace `str` or `none`

Namespace override. none uses the engine namespace.

Default: none

6.1.5 array-tree

Render the parse tree for a Typst math expression.

This is a diagnostic view of the tree consumed by parse; it does not create a Symbolica atom.

```
#array-tree($(x + 1)^2$)
```

```
(
  strong(body: raw(text: "attach")),
  (
    styled(child: raw(text: "base"), ..),
    (
      strong(body: raw(text: "()")),
      (
        styled(child: raw(text: "expr"), ..),
        (
          strong(body: raw(text: "add")),
          equation(body: [x]),
          equation(body: [1]),
        ),
      ),
    ),
  ),
  (
    styled(child: raw(text: "t"), ..),
    equation(body: [2]),
  ),
)
```

Parameters

```
array-tree(
  eqn: content,
  grammar: dictionary none
) -> content
```

eqn `content`

Math content to inspect.

grammar `dictionary` or `none`

Parser grammar override. none uses the engine grammar.

Default: none

6.1.6 canonical

Render an atom or matrix payload as Symbolica source text.

```
#raw(canonical(parse($x + 1$)))
```

```
1+x
```

Parameters

```
canonical(
  expr: bytes content int float str,
  namespaces: bool
) -> str
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload. Other supported expression values are first converted to expressions; strings represent numbers or symbol names.

namespaces `bool`

Include symbol namespaces in the output.

Default: `false`

6.1.7 inspect

Inspect scalar expression bytes as a semantic tree.

Returns `(ok: true, tree: ...)` on success, or `(ok: false, error: ...)` for a wrong input type, incompatible payload, or unsupported matrix. This reads an existing expression; use `parse` first for content or strings. Native payloads must come from a trusted, compatible plugin.

The versioned tree contains `root` and `attachments`. Nodes have a `kind`: `rational`, `complex`, `float`, `coefficient`, `symbol`, `call`, `sum`, `product`, or `power`. Sums have `terms`, products have `factors`, powers have `base` and `exponent`, and calls have `head` and `arguments`. This is normalized algebra, independent of Parsely and display notation. Exact rational numerators and denominators are decimal strings. Float nodes expose approximate `real` and `imaginary` values and retain an exact `payload`; other coefficient domains retain an opaque `payload`. Symbol descriptors retain names, namespaces, tags, attributes, and payloads.

```
#let result = inspect(parse($x^2$))
#assert(result.ok)
#assert.eq(result.tree.root.kind, "power")
#to-typst(from-tree(result.tree))
```

$$x^2$$

Parameters

```
inspect(value: any) -> dictionary
```

value `any`

A value that may contain Symbolica scalar expression bytes.

6.1.8 from-tree

Rebuild an expression from a semantic tree returned by `inspect`.

Edit algebraic children or exact rational values, and preserve the envelope and attachments. Symbol descriptors and float approximations must still agree with their payloads. Invalid trees report an error. Symbolica normalizes the result, so the returned expression may combine or reorder terms. Labels and unknown attachments survive the conversion.

Parameters

```
from-tree(tree: dictionary) -> bytes
```

tree `dictionary`

A version 1 Symbolica expression-tree dictionary.

6.1.9 notation

Build a document-side Atom notation layer.

Exact `heads` entries change one namespaced symbol. Exact `calls` entries receive the complete function node and its arguments. `tags` and `classes` provide broader fallbacks. Renderer closures receive a context dictionary; exact Atom metadata is attached by `to-typst` after a renderer returns.

Parameters

```
notation(
  heads,
  calls,
  tags,
  classes,
  fallback-head,
  fallback-call,
  fallback-node,
  fallback
) -> dictionary
```

6.1.10 merge-notation

Merge notation layers from least to most specific.

Parameters

```
merge-notation(..layers) -> dictionary
```

6.1.11 to-typst-source

Render an atom or matrix payload as Typst math source.

```
#raw(to-typst-source(parse($x + 1$)))
```

```
1+x
```

Parameters

```
to-typst-source(expr: bytes content int float str) -> str
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload, or a supported expression value.

6.1.12 to-typst

Render an atom or matrix payload as evaluated Typst math content.

Symbolica exports a generic algebra tree; Typst constructs its visual math directly from that tree. Ordinary calls retain an exact annotated head and visible arguments. A custom full-call renderer receives the complete node, and the framework attaches the exact call metadata after it returns. Matrix output remains display-only.

```
#to-typst(parse($x + 1$))
```

$1 + x$

Parameters

```
to-typst(
  expr: bytes content int float str,
  notation: dictionary none,
  block: bool
) -> content
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload, or a supported expression value.

notation `dictionary` or `none`

Per-call document notation layered over the engine's default. `none` uses that default unchanged.

Default: `none`

block `bool`

Render as a block equation instead of inline math.

Default: `false`

6.1.13 to-latex

Render an atom or matrix payload as LaTeX source.

```
#raw(to-latex(parse($x^2 + 1$)))
```

```
1+x^{2}
```

Parameters

```
to-latex(expr: bytes content int float str) -> str
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom or matrix payload, or a supported expression value.

6.1.14 to-float

Approximate numerical coefficients and built-in functions as decimals.

Variables remain symbolic. The WebAssembly build supports between 1 and 16 significant decimal digits. Use `evaluate` on the original expression when you need a numerical value rather than a symbolic display form. Expressions containing built-in calls with very large numeric arguments remain exact.

For decimal output, use `canonical` or `to-typst-source`. The structured `to-typst` renderer can currently omit digits or misplace the decimal point.

```
#let (parse, to-float) = init(namespace: "symbolica")
#let expr = parse($x + pi$)
#let approximate = to-float(expr, decimal-prec: 6)
#raw(canonical(approximate))
```

```
x+3.14159
```

Parameters

```
to-float(
  expr: bytes content int float str,
  decimal-prec: int
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to approximate.

decimal-prec `int`

Number of significant decimal digits to retain, from 1 through 16.

Default: 16

6.2 Built-in special functions

6.2.1 gamma-function

Construct the built-in gamma function.

Returns exact annotated math content, usable directly inside equations or as input to algebra and evaluation. The head is always `symbolica::gamma`; arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. Use `evaluate` for a numerical value, or `to-float` for an approximate symbolic expression.

```
#to-typst(gamma-function(5))
```

```
24
```

Parameters

```
gamma-function(z: bytes content int float str) -> content
```

z `bytes` or `content` or `int` or `float` or `str`

Function argument.

6.2.2 polygamma

Construct the built-in polygamma function of order `n`.

Returns exact annotated math content, usable directly inside equations or as input to algebra and evaluation. The head is always `symbolica::polygamma`; arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. Use `evaluate` for a numerical value, or `to-float` for an approximate symbolic expression.

```
#to-typst(polygamma(1, 1))
```

$$\frac{1}{6}\pi^2$$

Parameters

```
polygamma(
  n: bytes content int float str,
  z: bytes content int float str
) -> content
```

n `bytes` or `content` or `int` or `float` or `str`

Nonnegative integer order; zero gives the digamma function.

z `bytes` or `content` or `int` or `float` or `str`

Function argument.

6.2.3 polylog

Construct the built-in polylogarithm of order `s`.

Returns exact annotated math content, usable directly inside equations or as input to algebra and evaluation. The head is always `symbolica::polylog`; arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. Use `evaluate` for a numerical value, or `to-float` for an approximate symbolic expression.

```
#to-typst(polylog(2, literal("x")))
```

$$\mathrm{polylog}(2, x)$$

Parameters

```
polylog(
  s: bytes content int float str,
  z: bytes content int float str
) -> content
```

s `bytes` or `content` or `int` or `float` or `str`

Polylogarithm order.

z `bytes` or `content` or `int` or `float` or `str`

Function argument.

6.2.4 zeta-function

Construct the built-in Riemann zeta function.

Returns exact annotated math content, usable directly inside equations or as input to algebra and evaluation. The head is always `symbolica::zeta`; arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. Use `evaluate` for a numerical value, or `to-float` for an approximate symbolic expression.

```
#to-typst(zeta-function(2))
```

$\frac{1}{6}\pi^2$

Parameters

```
zeta-function(s: bytes content int float str) -> content
```

s `bytes` or `content` or `int` or `float` or `str`

Function argument.

6.2.5 bessel-j

Construct the built-in Bessel function of the first kind.

Returns exact annotated math content, usable directly inside equations or as input to algebra and evaluation. The head is always `symbolica::bessel_j`; arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. Use `evaluate` for a numerical value, or `to-float` for an approximate symbolic expression.

```
#let value = evaluate(bessel-j(0, 1))
$ J_0(1) approx #calc.round(value.re, digits: 8) $
```

$J_0(1) \approx 0.76519769$

Parameters

```
bessel-j(
  nu: bytes content int float str,
  z: bytes content int float str
) -> content
```

nu `bytes` or `content` or `int` or `float` or `str`

Bessel function order.

z `bytes` or `content` or `int` or `float` or `str`

Function argument.

6.2.6 bessel-y

Construct the built-in Bessel function of the second kind.

Returns exact annotated math content, usable directly inside equations or as input to algebra and evaluation. The head is always `symbolica::bessel_y`; arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. Use `evaluate` for a numerical value, or `to-float` for an approximate symbolic expression.

```
#let value = evaluate(bessel-y(0, 1))
$ Y_0(1) approx #calc.round(value.re, digits: 8) $
```

$Y_0(1) \approx 0.08825696$

Parameters

```
bessel-y(
  nu: bytes content int float str,
```

```
  z: bytes content int float str
) -> content
```

nu bytes or content or int or float or str

Bessel function order.

z bytes or content or int or float or str

Function argument.

6.2.7 bessel-i

Construct the built-in modified Bessel function of the first kind.

Returns exact annotated math content, usable directly inside equations or as input to algebra and evaluation. The head is always `symbolica::bessel_i`; arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. Use `evaluate` for a numerical value, or `to-float` for an approximate symbolic expression.

```
#let value = evaluate(bessel-i(0, 1))
$ I_0(1) approx #calc.round(value.re, digits: 8) $
```

$$I_0(1) \approx 1.26606588$$

Parameters

```
bessel-i(
  nu: bytes content int float str,
  z: bytes content int float str
) -> content
```

nu bytes or content or int or float or str

Bessel function order.

z bytes or content or int or float or str

Function argument.

6.2.8 bessel-k

Construct the built-in modified Bessel function of the second kind.

Returns exact annotated math content, usable directly inside equations or as input to algebra and evaluation. The head is always `symbolica::bessel_k`; arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. Use `evaluate` for a numerical value, or `to-float` for an approximate symbolic expression.

```
#let value = evaluate(bessel-k(0, 1))
$ K_0(1) approx #calc.round(value.re, digits: 8) $
```

$$K_0(1) \approx 0.42102444$$

Parameters

```
bessel-k(
  nu: bytes content int float str,
```

```
    z: bytes content int float str
) -> content
```

nu bytes or content or int or float or str

Bessel function order.

z bytes or content or int or float or str

Function argument.

6.3 Algebra and calculus

6.3.1 simplify

Validate and normalize an expression payload.

Parsing and algebra operations already combine equal terms and remove factors of one. `simplify` does not search for a shorter equivalent formula: use `expand`, `factor`, or `cancel-factors` for those transformations.

```
#let expr = parse($(x + 1)^2$)
#to-typst(simplify(expr))
```

$(1 + x)^2$

Parameters

```
simplify(expr: bytes content int float str) -> bytes
```

expr bytes or content or int or float or str

Atom payload or supported expression value to normalize.

6.3.2 expand

Multiply out products and nonnegative integer powers of sums.

Use expansion to compare polynomial expressions: expand their difference and check whether the result is zero.

```
#let factored = parse($(x - 1)(x + 1)$)
#let expanded = parse($x^2 - 1$)
#to-typst(expand(subtract(factored, expanded)))
```

0

Parameters

```
expand(expr: bytes content int float str) -> bytes
```

expr bytes or content or int or float or str

Atom payload or supported expression value to expand.

6.3.3 factor

Write a polynomial as a product of exact factors.

By default, factors have rational coefficients: $x^2 + 1$ stays irreducible. `complex: true` also permits rational multiples of the imaginary unit.

```
#let expr = parse($x^4 - 1$)
#to-typst(factor(expr))
```

$$(-1+x)(1+x)(1+x^2)$$

Parameters

```
factor(
  expr: bytes content int float str,
  complex: bool,
  square-free: bool
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Atom payload or supported expression value to factor.

complex `bool`

Factor over the complex rationals instead of the rationals. Cannot be combined with `square-free`.

Default: `false`

square-free `bool`

Return a square-free factorization, preserving multiplicities without fully splitting every factor. Complex input is handled over the complex rationals. Cannot be combined with `complex`.

Default: `false`

6.3.4 together

Write a rational expression over a common denominator.

```
#to-typst(together(parse($1/x + 1/y$)))
```

$$\frac{x+y}{xy}$$

Parameters

```
together(expr: bytes content int float str) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression whose rational terms should be combined.

6.3.5 cancel-factors

Cancel common factors between numerators and denominators.

Parts of the expression without a cancellation are unchanged. Canceling a factor can hide a point excluded by the original formula.

```
#to-typst(cancel-factors(parse($(x^2 - 1)/(x - 1)$)))
```

$$1+x$$

Parameters

```
cancel-factors(expr: bytes content int float str) -> bytes
```

expr bytes or content or int or float or str

Rational expression in which to cancel common factors.

6.3.6 apart

Decompose a rational expression into partial fractions.

Denominators are decomposed in the given indeterminate.

```
#let x = literal("x")
#to-typst(apart(parse($(2 x + 3)/((x + 1)(x + 2))$), x))
```

$$\frac{1}{1+x} + \frac{1}{2+x}$$

Parameters

```
apart(
  expr: bytes content int float str,
  var: bytes content str
) -> bytes
```

expr bytes or content or int or float or str

Rational expression to decompose.

var bytes or content or str

Indeterminate for the decomposition.

6.3.7 collect

Group terms with the same powers of selected variables or functions.

Other symbols remain in the coefficients. For example, collecting in x writes $ax + bx$ as $(a + b)x$ without assigning values to a or b.

```
#let x = literal("x")
#let expr = parse($a x + b x + a x^2 + b x^2$)
#to-typst(collect(expr, x))
```

$$x(a+b) + x^2(a+b)$$

Parameters

```
collect(
  expr: bytes content int float str,
  variables: bytes content str array
) -> bytes
```

expr bytes or content or int or float or str

Expression whose terms should be collected.

variables bytes or content or str or array

One variable or function, or an array of them.

6.3.8 coefficient

Extract the coefficient of a literal monomial or subexpression.

The result may depend on other symbols. Expand first when the desired monomial is still inside a product or power.

```
#let expr = expand(parse($(x + y)^3$))
#to-typst(coefficient(expr, parse($x^2$)))
```

$3y$

Parameters

```
coefficient(
  expr: bytes content int float str,
  monomial: bytes content int float str
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to inspect.

monomial `bytes` or `content` or `int` or `float` or `str`

Literal monomial or subexpression whose coefficient is wanted.

6.3.9 coefficient-list

Return collected key–coefficient pairs for one or more indeterminates.

Each result is `(key, coefficient)`, both as atom payloads. A key of `1` carries terms not polynomially collected in the requested variables. A coefficient that vanishes only through a deeper identity may remain.

```
#let x = literal("x")
#let pairs = coefficient-list(parse($x^2 + 5 x + 7$), x)
#pairs.map(
  pair => [#to-typst(pair.at(0)): #to-typst(pair.at(1))],
).join[, ]
```

$1: 7,\ x: 5,\ x^2: 1$

Parameters

```
coefficient-list(
  expr: bytes content int float str,
  variables: bytes content str array
) -> array
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to inspect.

variables `bytes` or `content` or `str` or `array`

One variable or function, or an array of them.

6.3.10 summands

Return the top-level additive terms of an expression.

This does not expand or recurse. A value that is not a sum is returned as a one-element array.

```
#summands(parse($x^2 + 2 x + 1$)).map(to-typst).join[, ]
```

$1,\ 2x,\ x^2$

Parameters

```
summands(expr: bytes content int float str) -> array
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to split into top-level summands.

6.3.11 indeterminates

Return the variables and function expressions that act as indeterminates.

Results are sorted in Symbolica's internal order.

```
#indeterminates(parse($f(x) + y$)).map(to-typst).join[, ]
```

$x,\ y,\ f(x)$

Parameters

```
indeterminates(
  expr: bytes content int float str,
  enter-functions: bool
) -> array
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to inspect.

enter-functions `bool`

Traverse function arguments as well as collecting the function itself.

Default: `true`

6.3.12 contains

Test whether an expression literally contains another expression.

The test follows the stored expression structure: for example, `x*y*z` contains `x`, but does not contain the regrouped product `x*y` as a node.

```
#contains(parse($x y z$), literal("x"))
```

`true`

Parameters

```
contains(
  expr: bytes content int float str,
  subexpression: bytes content int float str
) -> bool
```

expr bytes or content or int or float or str

Expression to search.

subexpression bytes or content or int or float or str

Literal subexpression to look for.

6.3.13 is-constant

Test whether an expression has no user-defined variables or functions.

Supported built-in functions such as `sin` and `cos` remain constant when all of their arguments are constant.

```
#is-constant(parse($cos(2) + 1/3$))
```

```
true
```

Parameters

```
is-constant(expr: bytes content int float str) -> bool
```

expr bytes or content or int or float or str

Expression to test.

6.3.14 derivative

Differentiate with respect to one variable, holding other symbols constant.

Apply `derivative` again for a higher derivative or a mixed partial derivative.

```
#let x = literal("x")
#let f = parse($x^3 y + y^2$)
$ (partial f)/(partial x) = #to-typst(derivative(f, x)) $
```

$$\frac{\partial f}{\partial x} = 3x^2y$$

Parameters

```
derivative(
  expr: bytes content int float str,
  var: bytes content str
) -> bytes
```

expr bytes or content or int or float or str

Expression to differentiate.

var bytes or content or str

Symbolica indeterminate, normally created with `literal`.

6.3.15 series

Compute a univariate series expansion around `expansion-point`.

The truncation depth is the rational number `depth / depth-denom`. With an absolute depth it is measured directly in `var`; with a relative depth it is measured from the lowest order encountered in the expression.

```
#let x = literal("x")
#let approximation = series(parse($sin(x)$), x, 0, 5)
#to-typst(approximation)
```

$$x - \frac{1}{6}x^3 + \frac{1}{120}x^5$$

Parameters

```
series(
  expr: bytes content int float str,
  var: bytes content str,
  expansion-point: bytes content int float str,
  depth: int,
  depth-denom: int,
  depth-is-absolute: bool
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to expand.

var `bytes` or `content` or `str`

Expansion variable, normally created with `literal`.

expansion-point `bytes` or `content` or `int` or `float` or `str`

Point about which to expand.

depth `int`

Numerator of the requested truncation depth.

depth-denom `int`

Denominator of the truncation depth.

Default: `1`

depth-is-absolute `bool`

Use an absolute depth when `true`, or a depth relative to the lowest order in the expression when `false`.

Default: `true`

6.3.16 integrate

Integrate an expression using the Rubi rules.

The variable must represent one symbol. Returns a best-effort antiderivative without an integration constant; unsupported parts remain unevaluated.

The first call to `integrate` or `integrate-with-steps` compiles and initializes the integration rule sets, which may take about 10 seconds. Both functions share the cached rules, so later calls and ordinary edits reuse them. Restarting the compiler or clearing its cache repeats this setup.

```
#to-typst(integrate(parse($x^2$), literal("x")))
```

$\frac{1}{3}x^3$

Parameters

```
integrate(
  expression: bytes content int float str,
  variable: bytes content str
) -> bytes
```

expression `bytes` or `content` or `int` or `float` or `str`

Integrand as an Atom payload or supported expression.

variable `bytes` or `content` or `str`

Integration variable as a symbol or Atom payload.

6.3.17 integrate-with-steps

Integrate an expression and return the nested Rubi rule trace.

The result contains `result` (Atom payload), `complete` (boolean), and `steps` (array). Each step contains `rule`, `depth`, `description`, `references`, `source`, `input`, and `output`. Render the input and output Atom payloads with `to-typst`. No integration constant is added.

The first call to `integrate` or `integrate-with-steps` compiles and initializes the integration rule sets, which may take about 10 seconds. Both functions share the cached rules, so later calls and ordinary edits reuse them. Restarting the compiler or clearing its cache repeats this setup.

Parameters

```
integrate-with-steps(
  expression: bytes content int float str,
  variable: bytes content str
) -> dictionary
```

expression `bytes` or `content` or `int` or `float` or `str`

Integrand as an Atom payload or supported expression.

variable `bytes` or `content` or `str`

Integration variable as a symbol or Atom payload.

6.4 Rewriting

6.4.1 rule

Build a reusable replacement rule for `replace-multiple`.

Captures from `pattern` are substituted into `rhs`. Store rules in an array to apply the same definitions to several expressions.

```
#let square = rule(parse("f(a_)"), parse("a_^2"))
#let shift = rule(parse("g(a_)"), parse("a_+1"))
#to-typst(replace-multiple(parse($f(x) + g(y)$), (square, shift)))
```

$1 + y + x^2$

Parameters

```
rule(
  pattern: bytes content int float str,
  rhs: bytes content int float str,
  non-greedy-wildcards: array,
  min-level: int,
  max-level: int none,
  level-range: array none,
  level-is-tree-depth: bool,
  partial: bool,
  allow-new-wildcards-on-rhs: bool,
  rhs-cache-size: int
) -> dictionary
```

pattern `bytes` or `content` or `int` or `float` or `str`

Pattern to match.

rhs `bytes` or `content` or `int` or `float` or `str`

Expression substituted for each match. Wildcards captured by `pattern` are substituted in this expression.

non-greedy-wildcards `array`

Wildcards that should prefer the smallest possible match.

Default: `()`

min-level `int`

Lowest expression level at which matching is allowed; the root is level zero.

Default: `0`

max-level `int` or `none`

Highest allowed matching level, or `none` for no upper bound.

Default: `none`

level-range `array` or `none`

Optional `(minimum, maximum)` pair overriding `min-level` and `max-level`; the maximum may be `none`.

Default: `none`

level-is-tree-depth `bool`

Count full expression-tree depth when `true`; otherwise levels increase when entering functions.

Default: `false`

partial `bool`

Allow a pattern to match part of a sum, product, or other term instead of requiring the whole term.

Default: `true`

allow-new-wildcards-on-rhs `bool`

Permit wildcards on `rhs` that do not occur in `pattern`. When `false`, such rules are rejected.

Default: `false`

rhs-cache-size `int`

Maximum number of substituted right-hand sides cached; use zero to disable this cache.

Default: `100`

6.4.2 replace

Replace subexpressions matching `pattern` with `rhs`.

A literal pattern replaces that exact expression, including occurrences inside function arguments. Add wildcards when the rule should also match related expressions. Matching uses the stored structure: expand first to find terms hidden inside a power such as $(x+1)^2$.

By default, non-overlapping outermost matches are replaced once. `repeat: true` reapplies the rule until nothing changes; cyclic rules do not terminate. Use `replace-multiple` to apply several rules together.

```
#let x = literal("x")
#to-typst(replace(parse($x^2 + f(x)$), x, parse($y + 1$)))
```

$f(1+y)+(1+y)^2$

Parameters

```
replace(
  expr: bytes content int float str,
  pattern: bytes content int float str,
  rhs: bytes content int float str,
  repeat: bool,
  once: bool,
  bottom-up: bool,
  nested: bool,
  non-greedy-wildcards: array,
  min-level: int,
  max-level: int none,
  level-range: array none,
  level-is-tree-depth: bool,
  partial: bool,
  allow-new-wildcards-on-rhs: bool,
  rhs-cache-size: int
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression in which to replace matches.

pattern `bytes` or `content` or `int` or `float` or `str`

Pattern to match.

rhs `bytes` or `content` or `int` or `float` or `str`

Replacement expression.

repeat `bool`

Reapply the rule until it makes no further change.

Default: `false`

once `bool`

Replace only the first match found during a pass.

Default: `false`

bottom-up `bool`

Visit deepest matches before outer matches.

Default: `false`

nested `bool`

Replace nested matches from the deepest outward, acting on the result of each inner replacement.

Default: `false`

non-greedy-wildcards `array`

Wildcards that should prefer the smallest possible match.

Default: `()`

min-level `int`

Lowest allowed matching level; the root is level zero.

Default: `0`

max-level `int` or `none`

Highest allowed matching level, or `none` for no upper bound.

Default: `none`

level-range `array` or `none`

Optional `(minimum, maximum)` pair overriding `min-level` and `max-level`.

Default: `none`

level-is-tree-depth `bool`

Count full tree depth instead of function-entry depth.

Default: `false`

partial `bool`

Allow matching a part of a term rather than the entire term.

Default: `true`

allow-new-wildcards-on-rhs `bool`

Permit `rhs` wildcards that are absent from `pattern`.

Default: `false`

rhs-cache-size `int`

Maximum number of substituted right-hand sides cached; zero disables the cache.

Default: `100`

6.4.3 replace-multiple

Apply several reusable replacement rules together.

The traversal options apply to the combined rule set. With `repeat: true`, the complete set is reapplied until no rule changes the expression; cyclic rule sets do not terminate.

```
#let r1 = rule(parse($f("a_")$), parse($h("a_")$))
#let r2 = rule(literal("x"), literal("z"))
#to-typst(replace-multiple(parse($f(x) + x$), (r1, r2)))
```

$$z + h(x)$$

Parameters

```
replace-multiple(
  expr: bytes content int float str,
  rules: array,
  repeat: bool,
  once: bool,
  bottom-up: bool,
  nested: bool
) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression in which to replace matches.

rules `array`

Array of dictionaries returned by `rule`.

repeat `bool`

Reapply the rule set until it makes no further change.

Default: `false`

once `bool`

Replace only the first match found during a pass.

Default: `false`

bottom-up `bool`

Visit deepest matches before outer matches.

Default: `false`

nested `bool`

Replace nested matches from deepest to outermost, acting on intermediate results.

Default: `false`

6.4.4 replace-wildcards

Substitute explicit values for wildcard placeholders in a pattern.

This does not search another expression. It transforms `pattern` itself and requires each key in `replacements` to be a wildcard symbol.

```
#to-typst(replace-wildcards(
  parse($k("a_")$),
  ((wild("a"), parse($x + 1$)),),
))
```

$$k(1 + x)$$

Parameters

```
replace-wildcards(
  pattern: bytes content int float str,
  replacements: array
) -> bytes
```

pattern `bytes` or `content` or `int` or `float` or `str`

Pattern containing wildcards to substitute.

replacements `array`

Array of `(wildcard, replacement)` pairs.

6.5 Evaluation and solving

6.5.1 evaluate

Evaluate one expression numerically with optional substitutions.

Supply a value for each unresolved variable or function call. Values may be real numbers or complex dictionaries `(re: number, im: number)`. A function-call key assigns a value to that call, not a definition for the function. Use `replace` to substitute a symbolic definition first.

The result is always `(re: float, im: float)`. Read `.re` for a known real result; use `to-float` instead if some variables should remain symbolic.

```
#let x = literal("x")
#let value = evaluate(parse($x^2 + f(2)$),
  values: ((x, 3.0), (parse($f(2)$), 4.0)))
#repr(value)
```

```
(re: 13.0, im: 0.0)
```

Parameters

```
evaluate(
  expr: bytes content int float str,
  values: array
) -> dictionary
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to evaluate.

values `array`

Array of `(expression, value)` substitution pairs. Values may be real numbers or `(re: ..., im: ...)` dictionaries.

Default: `()`

6.5.2 domain

Describe one real sampling axis for `evaluate-grid`.

Both endpoints are included when `samples` is greater than one. A single-sample domain evaluates only at `min`. The returned dictionary has the exact shape `(min: float, max: float, samples: int)`.

```
#repr(domain(-1, 1, samples: 3))
```

```
(min: -1.0, max: 1.0, samples: 3)
```

Parameters

```
domain(
  min: int float,
  max: int float,
  samples: int
) -> dictionary
```

min `int` or `float`

Finite lower endpoint; it must be less than `max`.

max `int` or `float`

Finite upper endpoint.

samples `int`

Number of evenly spaced points. Must be positive.

Default: `200`

6.5.3 evaluate-many

Evaluate one or more expressions at explicit points in one batch.

A single expression or variable may be passed directly; otherwise use an array. Every point must supply one real or complex value per variable. The result has one row per point in input order and one `(re: float, im: float)` dictionary per expression in expression order.

```
#let x = literal("x")
#let f = parse($x^3 - x$)
#let slope = derivative(f, x)
#let rows = evaluate-many((f, slope), x, (0, 1, 2))
#repr(rows.map(row => row.map(value => value.re)))
```

```
((-0.0, -1.0), (0.0, 2.0), (6.0, 11.0))
```

Parameters

```
evaluate-many(
  expressions: bytes content array int float str,
  variables: bytes content array str,
  points: array
) -> array
```

expressions `bytes` or `content` or `array` or `int` or `float` or `str`

One expression or a non-empty array of expressions to evaluate.

variables `bytes` or `content` or `array` or `str`

One variable or an array defining input-column order.

points `array`

Array of input rows. Each row is an array ordered like `variables`; for one variable, a scalar point is also accepted.

6.5.4 evaluate-grid

Evaluate expressions over a Cartesian product of real domains in one batch.

The result is `(shape: array, points: array, values: array)`. `shape` lists the sample count of every domain. The grid axes are flattened into rows with the last domain varying fastest: each `points` row contains real coordinates in variable order, while the corresponding `values` row contains one `(re: float, im: float)` dictionary per expression.

```
#let x = literal("x")
#let y = literal("y")
#let grid = evaluate-grid(
  parse($x^2 + y$), (x, y),
  (domain(-1, 1, samples: 3), domain(0, 1, samples: 2)),
)
shape: #repr(grid.shape); values: #repr(grid.values)
```

```
shape: (3, 2) values: (
  ((re: 1.0, im: 0.0),),
  ((re: 2.0, im: 0.0),),
  ((re: 0.0, im: 0.0),),
  ((re: 1.0, im: 0.0),),
  ((re: 1.0, im: 0.0),),
  ((re: 2.0, im: 0.0),),
)
```

Parameters

```
evaluate-grid(
  expressions: bytes content array int float str,
  variables: bytes content array str,
  domains: array
) -> dictionary
```

expressions `bytes` or `content` or `array` or `int` or `float` or `str`

One expression or a non-empty array of expressions.

variables `bytes` or `content` or `array` or `str`

One variable or an array defining grid-axis order.

domains `array`

One `domain` dictionary per variable, in the same order.

6.5.5 solve

Solve a linear or supported polynomial nonlinear system exactly.

Each expression in `system` is understood to equal zero. Polynomial systems are solved exactly using Gröbner bases and algebraic roots; coefficients may contain symbolic parameters when Symbolica can treat them rationally. `domain` may be `"complex"`, `"real"`, `"rational"`, or `"integer"`. The domain also supplies local assumptions for unrestricted external parameters: for example, `"real"` treats them as real while solving, without changing their global symbol properties.

The result is a dictionary with `branches`, the requested `variables`, inferred external `parameters`, and `domain`. `coverage` is `"complete"` or `"generic"`: generic results cover only parameter values where every expression in `coverage-guard` is nonzero. Other parameter values may have additional solutions. An empty `branches` array proves there are no solutions only when coverage is complete.

Each branch has a `values` array in the requested variable order. Free variables appear as themselves in that array and are also listed in `free-variables`. `conditions` retain restrictions: `"zero"`, `"nonzero"`, and `"positive"` carry an `expression`; `"domain-membership"` carries a `variable`, `value`, and `domain`. `conditional` reports whether these

restrictions remain; `point` means there are neither free variables nor branch conditions. The result's coverage guard still applies even when a branch is a point.

Branches also provide `domain`, `dimension`, and `codimension` (the number of requested variables minus the dimension). The result's `dimension` is the largest branch dimension, or `-1` for the complete empty set. Dimensions are `none` when Symbolica cannot establish them; they are not inferred by counting free variables.

```
#let x = literal("x")
#let y = literal("y")
#let solutions = solve(
  ($x^2 + y^2 - 5$, $y - 2x$).map(parse),
  (x, y), domain: "real",
)
#for branch in solutions.branches {
  let (x, y) = branch.values.map(to-typst)
  $ x = #x, quad y = #y $
}
```

$$x = -1, \quad y = -2$$

$$x = 1, \quad y = 2$$

Parameters

```
solve(
  system: array content int float str,
  variables: array content str,
  domain: str
) -> dictionary
```

system `array` or `content` or `int` or `float` or `str`

Expressions understood to equal zero.

variables `array` or `content` or `str`

Variables to solve for, in `values` order.

domain `str`

Exact solution domain: `"complex"`, `"real"`, `"rational"`, or `"integer"`.

Default: `"complex"`

6.5.6 nsolve

Find a real root of a univariate expression with Newton's method.

The expression is interpreted as equal to zero and evaluated with `f64` arithmetic. Convergence is local and depends on `init`; failure to converge within `max-iterations` produces an error.

```
#let x = literal("x")
#repr(nsolve(parse($x^2 - 2$), x, 1.0))
```

```
1.4142135623746899
```

Parameters

```
nsolve(
  expr: bytes content int float str,
  var: bytes content str,
  init: int float,
  prec: int float,
```

```
  max-iterations: int
) -> float
```

expr bytes or content or int or float or str

Expression understood to equal zero.

var bytes or content or str

Real solve variable, normally created with `literal`.

init int or float

Initial real guess.

prec int or float

Numerical tolerance for the Newton iteration.

Default: `1e-4`

max-iterations int

Maximum number of Newton iterations.

Default: `1000`

6.5.7 nsolve-system

Find a common real root of a system with multivariate Newton iteration.

Every expression is interpreted as equal to zero. `variables` and `init` must have matching lengths, and the returned floats follow that same order. Convergence is local and is not guaranteed for an arbitrary initial guess.

```
#let x = literal("x")
#let y = literal("y")
#repr(nsolve-system(
  (parse($x^2 + y - 3$), parse($x - y$)),
  (x, y), (1.0, 1.0),
))
```

```
(1.3027756377319948, 1.3027756377319948)
```

Parameters

```
nsolve-system(
  system: array content int float str,
  variables: array content str,
  init: array,
  prec: int float,
  max-iterations: int
) -> array
```

system `array` or `content` or `int` or `float` or `str`

Expressions understood to equal zero.

variables `array` or `content` or `str`

Variables to solve for, in result order.

init `array`

Initial real value for every variable.

prec `int` or `float`

Numerical tolerance for the Newton iteration.

Default: `1e-4`

max-iterations `int`

Maximum number of Newton iterations.

Default: `1000`

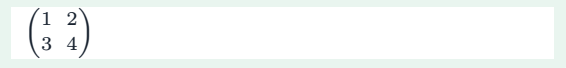
6.6 Matrices

6.6.1 matrix

Convert Typst values into an opaque Symbolica matrix payload.

Accepted forms are Typst math `mat(...)` or `vec(...)`, a non-empty rectangular nested array, a flat array interpreted as a column vector, a scalar interpreted as a one-by-one matrix, or an existing payload. Every entry must be convertible to a rational polynomial; general transcendental expressions are not valid matrix entries.

```
#to-typst(matrix($mat(1, 2; 3, 4)$))
```



Parameters

`matrix(value: bytes content array int float str) -> bytes`

value `bytes` or `content` or `array` or `int` or `float` or `str`

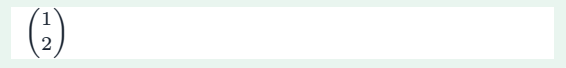
Matrix source value.

6.6.2 vector

Convert a non-empty sequence into a column-vector matrix payload.

Array entries must be rational-polynomial compatible. Typst math `vec(...)` is accepted directly; existing matrix bytes pass through unchanged.

```
#to-typst(vector((1, 2)))
```



Parameters

```
vector(values: array content bytes) -> bytes
```

values `array` or `content` or `bytes`

Vector entries, Typst `vec(...)` content, or an existing matrix payload.

6.6.3 identity

Create an `n` by `n` identity matrix payload.

```
#to-typst(identity(2))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Parameters

```
identity(n: int) -> bytes
```

n `int`

Positive matrix dimension.

6.6.4 eye

Create a square diagonal matrix from a non-empty entry array.

Every entry must be rational-polynomial compatible.

```
#to-typst(eye((1, 2)))
```

$$\begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$$

Parameters

```
eye(diag: array) -> bytes
```

diag `array`

Diagonal entries in top-left to bottom-right order.

6.6.5 matrix-add

Add two matrices entry by entry.

The matrices must have equal shapes. Matrix source values accepted by `matrix` may be supplied directly.

```
#to-typst(matrix-add(matrix(((1, 2), (3, 4))), identity(2)))
```

$$\begin{pmatrix} 2 & 2 \\ 3 & 5 \end{pmatrix}$$

Parameters

```
matrix-add(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Left matrix.

rhs `bytes` or `content` or `array` or `int` or `float` or `str`

Right matrix of the same shape.

6.6.6 matrix-sub

Subtract two matrices entry by entry.

The matrices must have equal shapes. Matrix source values accepted by `matrix` may be supplied directly.

```
#to-typst(matrix-sub(matrix(((1, 2), (3, 4))), identity(2)))
```

$$\begin{pmatrix} 0 & 2 \\ 3 & 3 \end{pmatrix}$$

Parameters

```
matrix-sub(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Left matrix.

rhs `bytes` or `content` or `array` or `int` or `float` or `str`

Right matrix of the same shape.

6.6.7 matrix-mul

Multiply two matrices, or multiply a matrix by a scalar expression.

Matrix multiplication requires compatible inner dimensions. A scalar must be rational-polynomial compatible. Because opaque bytes are interpreted as matrix payloads in the right-hand position, pass scalar atoms as ordinary values or math content rather than as preconstructed atom bytes.

```
#to-typst(matrix-mul(matrix(((1, 2), (3, 4))), identity(2)))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
matrix-mul(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Left matrix.

rhs `bytes` or `content` or `array` or `int` or `float` or `str`

Right matrix, or scalar Typst value/math content.

6.6.8 matrix-div-scalar

Divide every matrix entry by a nonzero scalar expression.

The scalar must be rational-polynomial compatible.

```
#to-typst(matrix-div-scalar(matrix(((2, 4), (6, 8))), 2))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
matrix-div-scalar(
  lhs: bytes content array int float str,
  rhs: bytes content int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix dividend.

rhs `bytes` or `content` or `int` or `float` or `str`

Nonzero scalar divisor.

6.6.9 transpose

Transpose a matrix, exchanging rows and columns.

```
#to-typst(transpose(matrix(((1, 2), (3, 4)))))
```

$$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Parameters

```
transpose(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to transpose.

6.6.10 determinant

Compute the exact determinant of a square matrix.

The result is an atom payload rather than a matrix payload.

```
#to-typst(determinant(matrix(((1, 2), (3, 4)))))
```

$$-2$$

Parameters

```
determinant(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Square matrix.

6.6.11 inv

Compute the exact inverse of an invertible square matrix.

Singular and non-square matrices produce an error.

```
#to-typst(inv(matrix(((1, 2), (3, 4)))))
```

$$\begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{pmatrix}$$

Parameters

```
inv(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Invertible square matrix.

6.6.12 matrix-solve

Solve the matrix equation `A x = b` exactly.

The row counts of `A` and `b` must agree. This strict solver reports an error when the system does not have the required unique solution; use `matrix-solve-any` for underdetermined systems.

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let b = vector((5, 1))
#to-typst(matrix-solve(A, b))
```

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Parameters

```
matrix-solve(
  A: bytes content array int float str,
  b: bytes content array int float str
) -> bytes
```

A `bytes` or `content` or `array` or `int` or `float` or `str`

Coefficient matrix.

b `bytes` or `content` or `array` or `int` or `float` or `str`

Right-hand-side matrix or column vector.

6.6.13 matrix-solve-any

Solve `A x = b` exactly, choosing one solution if underdetermined.

The row counts of `A` and `b` must agree. An inconsistent system still produces an error.

```
#let A = matrix($mat(2, 1; 1, -1)$)
#let b = vector((5, 1))
#to-typst(matrix-solve-any(A, b))
```

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Parameters

```
matrix-solve-any(
  A: bytes content array int float str,
  b: bytes content array int float str
) -> bytes
```

A `bytes` or `content` or `array` or `int` or `float` or `str`

Coefficient matrix.

b `bytes` or `content` or `array` or `int` or `float` or `str`

Right-hand-side matrix or column vector.

6.6.14 row-reduce

Row-reduce a matrix exactly using Gaussian elimination.

Returns `(matrix: bytes, rank: int)`. By default every column may contain a pivot. Set `max-col` to limit pivot search to the first `max-col` columns, which is useful for an augmented matrix whose trailing columns are right-hand sides; all columns are still transformed by the row operations.

```
#let rr = row-reduce(matrix(((1, 2), (3, 4))))
rank #rr.rank: #to-typst(rr.matrix)
```

rank 2: $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

Parameters

```
row-reduce(
  matrix: bytes content array int float str,
  max-col: int none
) -> dictionary
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to reduce.

max-col `int` or `none`

Number of leading columns eligible for pivots. `none` uses every column. An explicit value must be between zero and the column count.

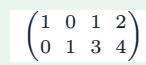
Default: `none`

6.6.15 augment

Horizontally concatenate two matrices as `[lhs rhs]`.

Both matrices must have the same number of rows.

```
#to-typst(augment(identity(2), matrix(((1, 2), (3, 4)))))
```



Parameters

```
augment(
  lhs: bytes content array int float str,
  rhs: bytes content array int float str
) -> bytes
```

lhs `bytes` or `content` or `array` or `int` or `float` or `str`

Left block.

rhs `bytes` or `content` or `array` or `int` or `float` or `str`

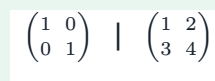
Right block with the same row count.

6.6.16 split-col

Split a matrix into left and right column blocks.

Returns exactly `(left, right)`, both as matrix payloads. The current matrix backend requires `index > 0` and `index < columns - 1`.

```
#let aug = augment(identity(2), matrix(((1, 2), (3, 4))))
#let parts = split-col(aug, 2)
#to-typst(parts.at(0)) | #to-typst(parts.at(1))
```



Parameters

```
split-col(
  matrix: bytes content array int float str,
  index: int
) -> array
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to split.

index `int`

Zero-based first column of the right block.

6.6.17 primitive-part

Divide a rational-polynomial matrix by its content.

The result is the primitive matrix whose coefficient GCD has been removed.

```
#let x = literal("x")
#let P = matrix(((mul(2, x), mul(4, x)), (mul(6, x), mul(8, x))))
#to-typst(primitive-part(P))
```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Parameters

```
primitive-part(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix whose entries are rational polynomials.

6.6.18 matrix-content

Compute the coefficient content of a rational-polynomial matrix.

The content is the common coefficient GCD and is returned as an atom payload.

```
#let x = literal("x")
#let P = matrix(((mul(2, x), mul(4, x)), (mul(6, x), mul(8, x))))
#to-typst(matrix-content(P))
```

$$2x$$

Parameters

```
matrix-content(matrix: bytes content array int float str) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix whose entries are rational polynomials.

6.6.19 matrix-at

Read one matrix entry as an atom payload using zero-based indices.

```
#let A = matrix(((1, 2), (3, 4)))
#to-typst(matrix-at(A, 0, 1))
```

$$2$$

Parameters

```
matrix-at(
  matrix: bytes content array int float str,
  row: int,
  col: int
) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to index.

row `int`

Zero-based row index. Must be within the matrix bounds.

col `int`

Zero-based column index. Must be within the matrix bounds.

6.6.20 matrix-shape

Return the matrix shape as the two-element array `(rows, columns)`.

```
#repr(matrix-shape(matrix(((1, 2), (3, 4)))))
```

```
(2, 2)
```

Parameters

```
matrix-shape(matrix: bytes content array int float str) -> array
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to inspect.

6.6.21 matrix-is-zero

Test exactly whether every matrix entry is zero.

This is an exact symbolic predicate, not a floating-point tolerance test.

```
#matrix-is-zero(matrix(((0, 0), (0, 0))))
```

```
true
```

Parameters

```
matrix-is-zero(matrix: bytes content array int float str) -> bool
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to inspect.

6.6.22 matrix-is-diagonal

Test exactly whether every off-diagonal matrix entry is zero.

Rectangular matrices are accepted; entries outside the main diagonal must be zero.

```
#matrix-is-diagonal(matrix(((1, 0), (0, 2))))
```

```
true
```

Parameters

```
matrix-is-diagonal(matrix: bytes content array int float str) -> bool
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Matrix to inspect.

6.6.23 matrix-derivative

Differentiate every matrix entry with respect to an indeterminate.

```
#let x = literal("x")
#to-typst(matrix-derivative(matrix(((pow(x, 2), x), (1, 0))), x))
```

$$\begin{pmatrix} 2x & 1 \\ 0 & 0 \end{pmatrix}$$

Parameters

```
matrix-derivative(
  matrix: bytes content array int float str,
  var: bytes content str
) -> bytes
```

matrix `bytes` or `content` or `array` or `int` or `float` or `str`

Rational-polynomial matrix to differentiate.

var `bytes` or `content` or `str`

Indeterminate with respect to which each entry is differentiated.

6.7 Scalar constructors

6.7.1 add

Construct an exact sum from expression values.

Arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. With no arguments, the result is the additive identity zero.

```
#to-typst(add("x", 1, "y"))
```

$$1 + x + y$$

Parameters

```
add(..terms: arguments) -> bytes
```

..terms `arguments`

Positional terms to add.

6.7.2 mul

Construct an exact product from expression values.

Arguments accept numbers, math content, expression payloads, or single-name strings. Use `parse` for expression strings. With no arguments, the result is the multiplicative identity one.

```
#to-typst(mul(2, "x", "y"))
```

$$2xy$$

Parameters

```
mul(..factors: arguments) -> bytes
```

..factors `arguments`

Positional factors to multiply.

6.7.3 neg

Negate an expression exactly.

```
#to-typst(neg("x"))
```

$$-x$$

Parameters

```
neg(expr: bytes content int float str) -> bytes
```

expr `bytes` or `content` or `int` or `float` or `str`

Expression to negate.

6.7.4 subtract

Subtract `rhs` from `lhs` exactly.

```
#to-typst(subtract("x", "y"))
```

$$x - y$$

Parameters

```
subtract(
  lhs: bytes content int float str,
  rhs: bytes content int float str
) -> bytes
```

lhs `bytes` or `content` or `int` or `float` or `str`

Minuend.

rhs `bytes` or `content` or `int` or `float` or `str`

Subtrahend.

6.7.5 divide

Construct the exact quotient `lhs / rhs`.

```
#to-typst(divide(1, "x"))
```

$$\frac{1}{x}$$

Parameters

```
divide(
  lhs: bytes content int float str,
  rhs: bytes content int float str
) -> bytes
```

lhs `bytes` or `content` or `int` or `float` or `str`

Numerator.

rhs `bytes` or `content` or `int` or `float` or `str`

Denominator.

6.7.6 pow

Construct the exact power `base ^ exp`.

```
#to-typst(pow("x", 3))
```

x^3

Parameters

```
pow(
  base: bytes content int float str,
  exp: bytes content int float str
) -> bytes
```

base `bytes` or `content` or `int` or `float` or `str`

Base expression.

exp `bytes` or `content` or `int` or `float` or `str`

Exponent expression.

6.8 Document helpers

6.8.1 logo

Draw the Symbolica logo inline with text.

The default size follows the surrounding font size. The logo is a vector image and stays sharp when enlarged.

```
Calculated with #logo() Symbolica.\
#logo(size: 2em)
```

Calculated with Σ Symbolica.



Parameters

```
logo(size: length) -> content
```

size `length`

Width and height of the logo.

Default: `1em`

6.9 Advanced configuration

6.9.1 init

Create an independent set of Symbolica functions.

The returned dictionary exposes Symbolica's parsing, algebra, evaluation, solving, integration, and matrix operations. Use `init` when you want to select a symbol namespace, plugin location, or parser grammar; ordinary calculations can use the imported top-level functions directly. Built-in special-function constructors always use Symbolica's function namespace; their arguments still use this engine's namespace and notation.

```
#let engine = init(namespace: "physics")
#let literal = engine.literal
#let render = engine.canonical
#raw(render(literal("x"), namespaces: true))
```

```
physics::x
```

Parameters

```
init(
  namespace: str,
  source: str bytes none,
  grammar: dictionary,
  notation: dictionary
) -> dictionary
```

namespace `str`

Default namespace for symbols parsed from Typst math or strings. A per-call `namespace` passed to `parse` or `literal` takes precedence.

Default: `"typst"`

source `str` or `bytes` or `none`

WebAssembly plugin path or bytes passed to Typst's `plugin` constructor. `none` selects the bundled engine. A custom source must be an uncompressed Wasm module; relative paths are resolved by Typst from this source file.

Default: `none`

grammar `dictionary`

Typst content grammar used by `parse` and `array-tree` unless they receive an explicit override. Expression strings always use Symbolica syntax.

Default: `_default_grammar`

notation `dictionary`

Default document-side notation used by `to-typst`. Build one with `notation`; a per-call override still takes precedence.

Default: `atom-render.notation()`

7 Namespaces and parser grammars

If you need separate groups of symbol names or a different way to read Typst math, use `init()` to configure an engine. It returns a dictionary of functions that share those settings. The directly imported functions use the default settings.

7.1 Keep symbol names separate

Suppose two models both use x, but for different quantities. A namespace adds a prefix to each symbol's internal name, such as `left::x` or `right::x`. These are different variables, even though both normally display as x. This is useful when combining calculations from different models or packages.

```
#let left = literal("x", namespace: "left")
#let right = literal("x", namespace: "right")
#let difference = subtract(left, right)

#raw(canonical(difference, namespaces: true))
```

```
left::x-right::x
```

Use `canonical(..., namespaces: true)` to inspect the full names. A namespace is part of a symbol's mathematical identity.

The imported functions use `"typst"` by default. Pass `namespace` to `parse`, `literal`, `function-head`, or `wild` for an individual call. For a calculation that uses one namespace throughout, set it once:

```
#let model = init(namespace: "oscillator")
#let parse = model.parse
#let literal = model.literal
#let x = literal("x")
#let f = parse($x^2 + 1$)

$ f(x) = #to-typst(f), quad f'(x) = #to-typst(derivative(f, x)) $
```

$$f(x) = 1 + x^2, \quad f'(x) = 2x$$

Here `parse` and `literal` both create `oscillator::x`. The algebra functions accept these expressions without changing their names. A per-call `namespace` overrides the engine's default, and creating another engine with the same namespace produces the same symbol identities.

In Typst math, a bare `$x$` does not read a `#let x` binding. The parser assigns it the current namespace. Interpolating `$#x$` uses the bound value and preserves its existing identity, even when the surrounding expression uses another namespace.

`"symbolica"` is the namespace for built-in functions. Special-function constructors such as `polylog` select it automatically. Their arguments keep their own symbol identities. Symbolica's namespace documentation explains how the same names are used from Python and Rust.

7.2 Define a parser grammar

A grammar tells the parser which operators to recognize and how to group their arguments. For example, suppose you want $\oplus$ to mean addition in your input:

```
#let grammar = (
  add: (infix: $⊕$, prec: 1, assoc: true),
)
#let engine = init(grammar: grammar)
#let expression = (engine.parse)($x ⊕ y ⊕ x$)

$ #to-typst(expression) $
```

$$2x + y$$

`infix` specifies the operator between arguments. `prec` sets its precedence relative to other operators: higher values bind more tightly. `assoc: true` collects a chain into one node. The rule name `add` tells the plugin to interpret that node as addition.

A supplied grammar **replaces** the default content grammar. This small example only defines $\oplus$; it does not retain the package's rules for multiplication, fractions, or function calls. Include rules for the syntax you want to accept. To support both $+$ and $\oplus$, use a second rule and rewrite its node to `add`:

```
#let grammar = (
  add: (infix: $+$, prec: 1, assoc: true),
  custom-plus: (
    infix: $⊕$, prec: 1, assoc: true,
    rewrite: node => { node.head = "add"; node },
  ),
)
#let expression = parse($x + y ⊕ x$, grammar: grammar)
$ #to-typst(expression) $
```

$$2x + y$$

Use `parse(..., grammar: grammar)` for one call or `init(grammar: grammar)` to reuse it. `array-tree(..., grammar: grammar)` displays the parsed structure and helps check how a formula was grouped.

The parser is Parsely. Its grammars support prefix and postfix operators, content patterns, guards, and rewrite functions. The resulting nodes must use the structure expected by the plugin: recognized heads such as `add` and `mul` perform algebra; other heads become symbolic function calls with their positional arguments. A new rule does not add a numerical implementation or differentiation rules for a new function.

Grammars apply to Typst content only. `parse("x^2 + y")` always uses Symbolica's string syntax, even on an engine with a custom grammar. Passing an explicit `grammar` to any non-content input is an error. For changes to the displayed output, use `notation` instead.

8 Compatibility and licensing

This manual describes the official Symbolica Typst plugin 0.2.0, using Symbolica 3.0. The package is tested with Typst 0.14 or newer.

The official `symbolica` plugin is free for all use within Typst, including academic and commercial work. Its Typst permission requires no payment, registration, or license key.

The original plugin source code is released under the MIT License. The bundled WebAssembly engine is included with redistribution permission. The underlying Symbolica computer algebra system is governed by its own license terms outside this Typst usage permission.

For source, issues, and release history, visit github.com/symbolica-dev/symbolica-typst-plugin.

9 Acknowledgements

The algebra engine is Symbolica. Integration uses Symbolica Integrate and the Rubi ruleset. The explanations and examples in this manual draw on the Symbolica documentation, adapted to the Typst interface.

Parsely parses Typst math content. Tidy generates the manual's examples and API reference.

Thanks to the contributors to these projects.